

MIR Suite

Technical documentation — architecture, operation and reference

Technical documentation • Version 2.0 • October 2026

Author	Francisco Javier Martínez Peña Universidad Autónoma del Estado de México (UAEM)
Supervisor	Prof. José Luis Martínez Lastra, FAST Lab, Tampere University
Applies to	Repository github.com/fjimp-dev/TUNI-PROJECT , release v1.0 (state of the system at the end of July 2026)
Audience	Whoever operates, maintains or extends the platform
Companion document	MIR Suite — Project Report, v2.0 (what was done and why; open items)
Credentials	This document contains no passwords or keys. They live only on the Jetson, in <code>config/.env</code> and <code>UI/backend/data/</code> .

Document control

Version	Date	Changes
1.0	24 Jul 2026	First complete version.
2.0	Oct 2026	Every statement re-checked against the code at release v1.0. Credentials removed. Corrected statements about the emergency stop, UR start-up checks, launch-patch scope, MiR authentication, reconnection back-off, TLS, privileges, MiR address defaults and start-up commands. Added: motion pipeline, DDS exposure, troubleshooting, testing, known limitations, full environment-variable list.

Contents

1. System overview
2. Architecture
3. MiR200 integration
4. UR5e integration
5. Camera
6. Web interface
7. Security
8. Operation
9. Troubleshooting
10. Testing
11. Known limitations
12. Appendices: environment variables, repository layout

1. System overview

The MIR Suite is the control software of the FAST Lab dual-arm mobile manipulator. It runs as a set of Docker containers on the onboard Jetson and is operated from a browser. It replaced an earlier setup in which every process was started by hand over SSH (original system by Eemil Rintala, in `pbd_system` and `pandai_ark`; those projects are not modified by the suite).

Table 1.1. Hardware and addresses on the robot network (192.168.1.0/24).

Component	Model	Address / interface
Onboard computer	NVIDIA Jetson AGX Orin 64 GB (Advantech MIC-733), hostname "Kevin"	192.168.1.75 (robot LAN); university uplink on lan1 (DHCP)
Mobile base	MiR200	192.168.1.14 (MIR_IP; see 3.4)
Left arm	Universal Robots UR5e	192.168.1.102
Right arm	Universal Robots UR5e	192.168.1.103
Hands	2 × BrainCo Revo1	USB serial /dev/ttyUSB0 (left), /dev/ttyUSB1 (right); order not guaranteed — a udev rule is recommended
Camera	Orbbec Gemini 335Lg (RGB-D)	USB
Force/torque	2 × Nordbo NRS-6200	192.168.1.112 (left), .113 (right); WebSocket :2003

Component	Model	Address / interface
Router	Teltonika RUTX50 (5G, Elisa SIM)	192.168.1.1; Wi-Fi SSIDs in ROUTER_MODEM/documentation (the MiR must use the 2.4 GHz one)

Software stack: Ubuntu 22.04 · ROS 2 Humble with FastDDS (`rmw_fastdds_cpp`) · Docker Compose · FastAPI/uvicorn (Python) · Svelte 5 + Vite · Caddy · Avahi (mDNS).

Safety notes for anyone commanding the arms

The arms are mounted in a reversed orientation on the torso: a positive joint delta does not always move the way you expect. Start every new motion with a delta of 0.01 rad or less and confirm it physically.

Do not move the wrist joints without an explicit reason (the API requires `allow_wrist`).

Do not restart the `mir_ur_driver` container while an arm is moving.

Do not switch the MiR to Running mode without someone ready to recover it (see 3.5).

The software emergency stop does not currently cancel arm motions (4.4). Use the hardware emergency stops.

The ROS 2 graph is reachable from the robot network without authentication (7). Do not connect untrusted devices to it.

2. Architecture

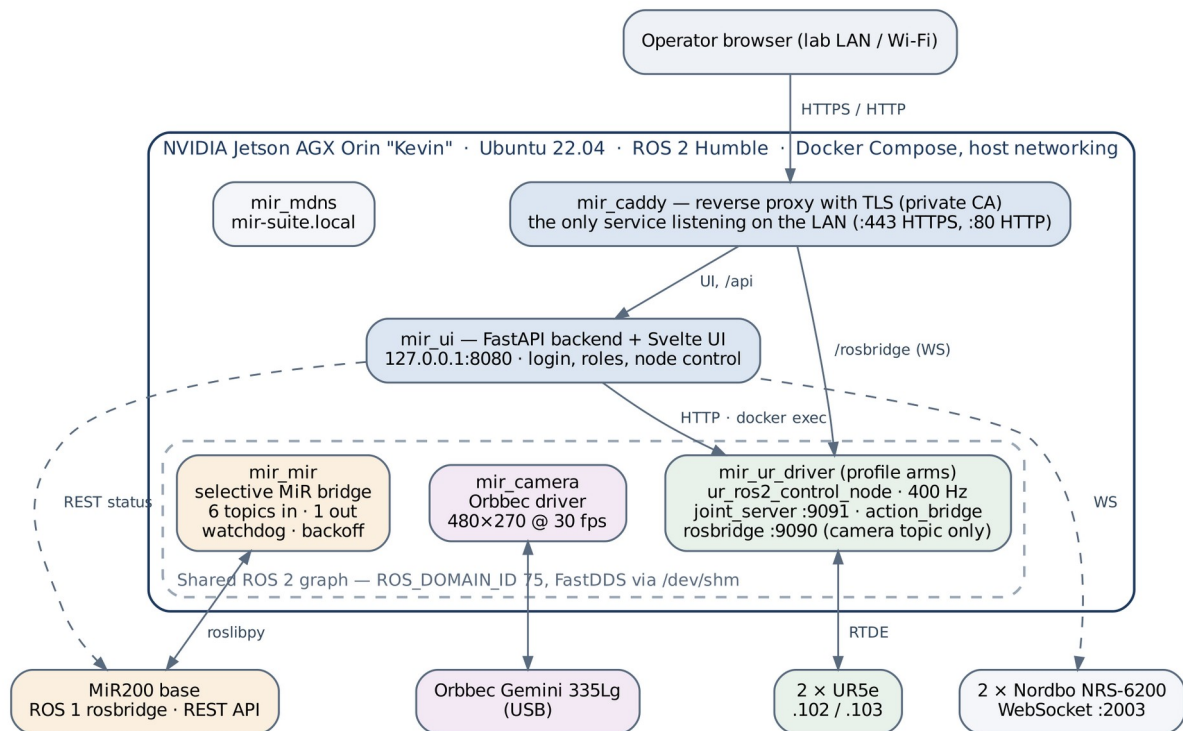


Figure 2.1. Services, data paths and hardware.

Table 2.1. Services defined in `docker-compose.yml`.

Service	Container	Profile	Role	Limits
caddy	mir_caddy	always	Reverse proxy and TLS; the only service listening on the LAN	256 MB · 0.5 CPU

Service	Container	Profile	Role	Limits
mdns	mir_mdns	always	Publishes mir-suite.local via Avahi	128 MB · 0.3 CPU
mir_ui	mir_ui	always	FastAPI backend and built web UI; controls the other containers	512 MB · 1 CPU
camera	mir_camera	always	Orbbec driver, started on demand as a node	1 GB · 1.5 CPU
mir	mir_mir	always	Selective MiR bridge, scanner merger, watchdog and liveness check	1 GB · 1 CPU
ur_driver	mir_ur_driver	arms, full	UR driver and controllers, rosbridge, joint server, action bridge, hand node	4 GB · 3 CPU
ur_driver_sim	mir_ur_driver_sim	sim	Same image against simulated hardware, for tests	4 GB · 3 CPU

Every service has a health check (the camera's reports unhealthy while no camera node is running, which is expected). Design decisions that a maintainer needs to know:

- **Host networking everywhere.** The Jetson kernel does not support Docker bridge networks (nor the iptables raw table, so images are built with `--network=host` through `build.sh`).
- **One ROS graph.** All ROS containers share `ROS_DOMAIN_ID=75`, so `ros2 topic list` shows the same ≈ 41 topics from any of them.
- **Shared memory is required.** FastDDS uses `/dev/shm` between processes on the same host; every ROS container mounts it (or `/dev`). Without it, topics are listed but carry no data ("topic exists, zero messages").
- **Nothing moves by default.** The arm container is in an opt-in profile, and even when it runs the arm driver is started only from the interface; logging in starts nothing.
- **rosbridge and the joint server live in the arm container.** The Camera view and the arm controls therefore need `mir_ur_driver` (or the sim twin) to exist. Create it once with `docker compose --profile arms up -d`. The sim and real containers share domain 75 and ports 9090/9091: never run both.
- **Self-contained workspace.** The suite mounts its own copy of the ROS workspace (`robot_workspace/`, not versioned) and overlays patches (for example `UR/duo_ur_launch_patch/`) instead of editing Emil's packages.

2.1 Privileges

No container runs with `privileged: true`. Access is granted per need:

Container	Grants	Caveat
mir_ur_driver	<code>SYS_NICE</code> , <code>IPC_LOCK</code> with <code>rtprio/memlock</code> limits (real-time loop); cgroup rules <code>c 188:*</code> (USB serial, hands) and <code>c 189:*</code>	Bind-mounts the host <code>/dev</code> (device nodes visible, access limited by the cgroup rules) and <code>./config</code> , which holds <code>.env</code> and the TLS keys.
mir_camera	<code>/dev/bus/usb</code> and cgroup rule <code>c 189:*</code> (libusb)	USB reset is done on the host by a root systemd path unit (<code>PERIPHERAL/camera_orbbec_usb_reset/</code> , installed with <code>install_host_helper.sh</code>), not from the container.
mir_mir	Host networking only	—

Container	Grants	Caveat
mir_ui	The Docker socket	Root-equivalent on the Jetson. Needed to start and stop nodes and containers.

3. MiR200 integration

3.1 Problem

The earlier link (`mir_raw.py`) connected to the MiR's internal ROS 1 `rosbridge` and republished every topic it offered — about 150, including internal diagnostics — into the ROS 2 domain used by the arm controllers. This coincided with MiR Error 9000 (safety PLC and emergency stop) when entering Running mode and with degraded arm communication.

3.2 Selective bridge

The bridge (`MiR/bridge_mir/mir_bridge.py`, design by Wael from the `fortis-uc1-tau / TK32` project, integrated on 10 July 2026) uses `roslibpy` and forwards only the following topics:

Direction	Topic	Rate	Purpose
MiR → ROS 2	<code>/f_scan, /b_scan</code>	≈13 Hz	Front and rear SICK safety laser scanners
MiR → ROS 2	<code>/odom</code>	see note	Base odometry
MiR → ROS 2	<code>/tf (map→odom removed), /tf_static</code>	—	Transforms; <code>map→odom</code> is dropped so a future local SLAM can provide it
MiR → ROS 2	<code>/MC/battery_percentage</code>	—	Battery level
ROS 2 → MiR	<code>/cmd_vel</code>	—	Base velocity (supervised by the MiR safety PLC)

Note: the `/odom` rate is recorded as ≈41 Hz in the operator manual and ≈3.5 Hz in the watchdog notes; re-measure with `ros2 topic hz /odom` before relying on it.

3.3 Supervision

Layer	Behaviour
Liveness (<code>mir_liveness.py</code>)	Subscribes to <code>/odom</code> and touches a heartbeat file on every message.
Watchdog (<code>mir_watchdog.sh</code>)	Checks the heartbeat every 15 s; after 25 s without data it terminates the bridge. A 60 s grace period follows each (re)start.
Reconnection (<code>mir_entrypoint.sh</code>)	Restarts the bridge. The wait doubles after each run shorter than 60 s (20 s, 40 s, then 60 s maximum) and resets to 10 s after a longer run.
Scanner merger (<code>scanners_merger.py</code>)	Runs outside the reconnection loop, independent of the MiR state.

3.4 Configuration

The MiR address comes from `MIR_IP` in `config/.env` (currently 192.168.1.14). **The defaults disagree:** the backend defaults to .14, while the bridge, watchdog and entrypoint default to .13 (the address of the earlier DHCP reservation). Set `MIR_IP` explicitly and align the defaults when the DHCP reservation is fixed. The backend reads status, battery and mode from `http://MIR_IP/api/v2.0.0/status` without credentials and caches the result. (Other MiR REST endpoints use HTTP Basic authentication of the form `base64(user:sha256(password))`; see `MiR/documentation/.`)

3.5 Known issues

- **Wi-Fi.** The base's link was stable (0 % packet loss) on the router's 2.4 GHz SSID with a DHCP reservation; after a later network change it must be moved back to that configuration.
- **Phantom obstacle stops.** Occasional protective stops attributed to an obstacle, consistent with brief interruptions of the rear scanner (loose cable or dirty window suspected; not yet inspected).
- **Clock.** The MiR clock is about ten years off (affects log timestamps only).
- **Error 9000 evidence.** Not observed from 10 to 27 July 2026 in normal use; no controlled long-duration test in Running mode has been run.

4. UR5e integration

4.1 Driver

A single `ur_ros2_control_node` drives both arms (`headless_mode=True`, `reverse_ip=192.168.1.75`, `tf_prefix` `left_/right_`) and publishes `/joint_states` at ≈ 400 Hz. The reverse interfaces use ports 50001–50004 (left) and 50011–50014 (right). The patched launch file in `UR/duo_ur_launch_patch/` is mounted only into the simulation container; the real driver uses the original launch file from the workspace.

4.2 Motion pipeline

1. The interface calls `POST /api/ur/move` with arm, joint and delta. The backend checks the control permission, rejects deltas above `UR_MAX_DELTA` (0.5 rad) and rejects wrist joints unless `allow_wrist` is set.
2. The backend runs `joint_mover.py` inside `mir_ur_driver` (docker exec), which sends a `FollowJointTrajectory` goal with **one** target point and duration $\max(3\text{ s}, 60\text{ s/rad} \times |\text{delta}|)$. A two-point trajectory with a stale start point caused velocity spikes that tripped the robot's safety speed limit; the single-point form avoids them.
3. After a successful result, the mover re-reads the joint. For deltas of 0.02 rad or more, if the joint moved less than half the delta it reports "no motion" (the UR external-control program had stopped). Smaller moves are not verified.
4. On "no motion" or a timeout the backend calls `resend_robot_program` for that arm and retries.
Caveat: the mover waits at most 15 s for the result, but a move longer than 0.25 rad lasts more than 15 s; the retry then resends the full delta from the partly moved position and could overshoot. The interface only sends ± 0.1 rad elbow jogs, so this has not occurred.

4.3 Start-up

`ur_start.sh` launches the driver and waits up to 45 s (`UR_ARMS_TIMEOUT`) for both arms to confirm their RTDE connection. If they do not, it logs a warning and **continues**, so check the log when the arms do not respond. If the driver is started with the arms powered off, the node aborts and leaves a stale lock; recover with `docker restart mir_ur_driver` and a single Start from the interface.

4.4 Payload, freedrive and emergency stop

- **Payload** (`/api/ur/payload`): mass and centre of gravity per arm.
- **Freedrive** (`/api/ur/freedrive`): switches the arm to the freedrive controller and back. The interface refuses to enable it until a payload above 1 kg has been set for that arm. This check is in the interface only and uses the configured, not a measured, payload.
- **Emergency stop** (`/api/estop`, red button in the top bar, behind a confirmation dialog). Intended to cancel active goals and disable freedrive. **As implemented it only disables freedrive:** the `stop_arm` message is built with incorrect quoting and rejected by `ros2 topic pub`; and even when fixed,

`stop_arm` cancels only goals sent through `action_bridge`, not jog moves from `joint_mover`. It is software only and never a substitute for the hardware emergency stops.

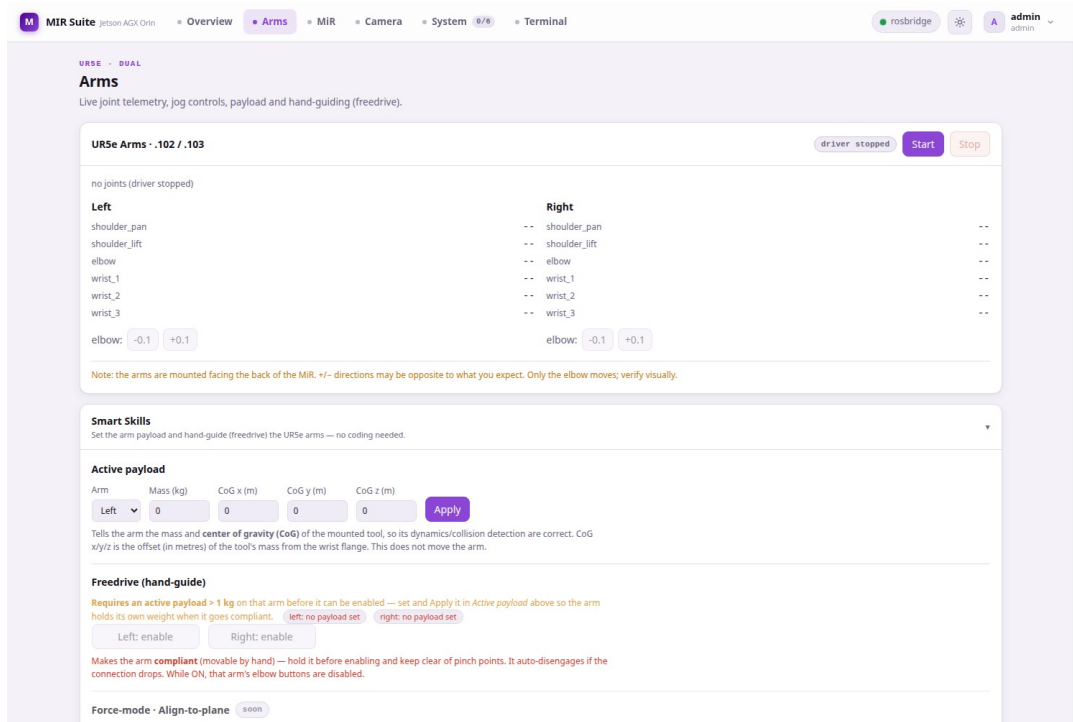


Figure 4.1. Arms view: joint telemetry, incremental moves, payload and freedrive.

5. Camera

The Orbbec driver runs in `mir_camera` and is started from the System view as one of two mutually exclusive nodes, because both use the same USB device:

Node	Launch parameters	Use
Camera (color)	color 480 × 270 @ 30 fps, depth disabled	Live view in the Camera tab
Camera (color + depth + cloud)	same colour stream, depth registered, coloured point cloud	Perception and grasping work

The browser receives `/camera/color/image_raw/compressed` through `rosbridge`; the topic is published at ≈ 30 Hz (the Camera view shows the rate actually received). The camera occasionally drops off the USB bus; the Camera view shows a USB status badge and a **Reset USB** button (`/api/camera/usb/reset`), and recovery is also attempted automatically.

6. Web interface

6.1 Backend

`UI/backend/main.py` is a single FastAPI file ($\approx 1,700$ lines) using `uvicorn`, `docker-py`, `httpx` and `WebSockets`. It listens on `127.0.0.1:8080` behind `Caddy` when `UI_BIND_HOST=127.0.0.1` is set in `config/.env` (the code default is `0.0.0.0`) and is bind-mounted into `mir_ui`, so backend changes need only `docker restart mir_ui`.

Table 6.1. API endpoints. All except /health and /api/login require a session token (header X-MIR-Token).

Area	Endpoints	Notes
Session	POST /api/login, POST /api/logout, GET /api/me	Token TTL 24 h; lockout after 8 failures for 300 s
Profile	PUT /api/me/config, POST /api/me/apply	Saved node selection per user
Users	GET/POST /api/users, PUT/DELETE /api/users/{username}	Administrators only
Containers	GET /api/containers, POST /api/containers/{name}/start stop	Whitelisted containers
Nodes	GET /api/nodes, POST /api/nodes/{id}/start stop, GET /api/nodes/{id}/logs	Per-node locks; pkill patterns validated
System	GET /api/system, GET /api/config, GET /health	/health is unauthenticated
Arms	GET /api/ur/status, /api/ur/joints; POST /api/ur/start, /stop, /move, /payload, /freedrive	Control actions need can_control
Safety	POST /api/estop	Software stop (4.4)
MiR	GET /api/mir/status	Proxies the MiR REST API, cached
Camera	GET /api/camera/usb, POST /api/camera/usb/reset	USB status and recovery
Force	GET /api/force, POST /api/force/{side}/tare connect disconnect	Nordbo sensors over WebSocket
Terminal	WebSocket /api/term	Admin token + SHELL_PASSWORD + whitelisted container

6.2 Frontend

Svelte 5 + Vite in UI/web/, compiled into the mir_ui image (frontend changes need an image rebuild). It has 11 components and 11 reactive stores (lib/*.svelte.js); cross-component state lives only in the stores.

Route	View	Content
#/overview	Overview	Service status, arm telemetry, force readings, process list, Record button
#/arms	Arms	Joint telemetry and incremental moves; payload and freedrive; force panel
#/mir	MiR	Base status, battery, mode, staleness
#/camera	Camera	Live colour stream, FPS, USB status and reset
#/system	System	Start/stop nodes and containers, logs, "what is this?" help per node
#/terminal	Terminal	Shell in a whitelisted container (administrators, second password)

6.3 Data flow

```

Browser — HTTPS :443 / HTTP :80 —> Caddy
├─ /rosbridge —> rosbridge 127.0.0.1:9090 (subscribe: camera topic only)
└─ everything else —> uvicorn 127.0.0.1:8080 (FastAPI)
    ├─ docker exec —> scripts inside the containers (moves, nodes)
    ├─ HTTP —> joint_server 127.0.0.1:9091 (joint states, cached 100 ms)
    ├─ HTTP —> MiR REST API (MIR_IP:80)
    └─ WebSocket —> Nordbo sensors (:2003)

```

Only the camera stream travels through rosbridge. Joint states are polled through the API, and every command goes through the authenticated API.

6.4 Users and roles

Profiles are YAML files in `UI/backend/data/profiles/` (git-ignored), with PBKDF2-SHA256 password hashes. Roles are `admin` (everything, including user management and the terminal) and `user`. A user also needs the `can_control` flag to start or stop anything or move the robot; without it the interface is read-only and the backend rejects control calls. Seed accounts: `admin`, `wael`, `pablo`. Their passwords were changed from the defaults on 24 July 2026 and are not recorded in this document; the backend logs a warning at start-up if any seed account still uses its default.

7. Security

Measure	Implementation
Web surface	Only Caddy serves web traffic on the LAN (80/443). <code>uvicorn</code> , <code>rosbridge</code> and <code>joint_server</code> listen on 127.0.0.1. DDS is not covered (see below).
rosbridge whitelist	Topics: <code>/camera/color/image_raw/compressed</code> only (the glob also permits publishing to it). Services, actions and parameters: denied. Arms cannot be commanded through it.
Authentication	PBKDF2-SHA256 (200,000 iterations); 24 h tokens in memory; lockout after 8 failures for 300 s; dummy hash for unknown users (timing).
Authorization	Roles <code>admin/user</code> plus <code>can_control</code> , enforced in the backend.
Web terminal	Admin token + <code>SHELL_PASSWORD</code> (constant-time comparison) + container whitelist. Disabled when <code>SHELL_PASSWORD</code> is empty or a known default. Black-box tested on 24 July 2026.
Command hardening	Regex whitelist for <code>pskill</code> patterns; per-node locks; Pydantic validation of request bodies; terminal resize capped.
TLS	Private root CA (10 years) signing the server certificate directly (825 days), generated by <code>config/tls/gen_cert.sh</code> . CA key only on the Jetson in <code>config/tls/</code> (mode 600, git-ignored). Names: <code>mir-suite.local</code> , <code>tunisuite.local</code> , <code>lab.local</code> , <code>localhost</code> , <code>192.168.1.75</code> , <code>127.0.0.1</code> .
Secrets	<code>.env</code> , <code>*.key</code> , <code>*.pem</code> and <code>UI/backend/data/</code> are git-ignored.

Accepted risks and gaps (details and recommendations in the project report, Section 6):

- **The ROS 2 graph is open to the robot network.** All containers use host networking with FastDDS in domain 75 and no DDS security, so any device on 192.168.1.0/24 can publish `/cmd_vel` (forwarded to the MiR) or send trajectories to the arm controllers without logging in. Restrict DDS to the Jetson (shared-memory or localhost-only transport, or an interface whitelist in a FastDDS profile) or enable SROS2.
- The software emergency stop does not cancel arm motion (4.4).
- Plain HTTP on port 80 is served alongside HTTPS so that machines without the lab CA can connect; passwords and commands then travel unencrypted on the lab Wi-Fi. Disable it in `UI/caddy/Caddyfile` once the CA is installed on operator machines (`/ca.crt` serves the certificate; `config/tls/trust_ca.sh` installs it).
- The `/rosbridge` route is not behind the login, so the camera stream is visible to anyone on the lab network.
- `mir_ui` holds the Docker socket (Section 2.1).

- Credentials were committed to the repository before release v1.0 (interface password, router administrator and SSH password, Wi-Fi key). They were removed from the current files in v1.0 but remain in the history: change them all.
- Factory credentials remain on some lab devices, including the MiR web interface.

8. Operation

8.1 Access

Address	Notes
https://mir-suite.local	Encrypted. Requires the lab CA on the client (download from /ca.crt).
http://192.168.1.75	Unencrypted; works on any device on the lab network.

8.2 Start and stop

Suite folder on the Jetson: /home/lab/Desktop/MIR/mir_suite. The desktop launcher (start_mir_suite.sh) starts the always-on services and opens the interface, or stops everything. From a terminal:

```
docker compose up -d          # always-on services (no arms)
docker compose --profile arms up -d  # + real arm driver container
docker compose --profile sim up -d   # + simulated arms (no hardware)
docker compose --profile arms --profile sim down  # stop everything
```

The launcher runs only the first command, so create the arm container once with the second (it starts rosbridge, the joint server and the action bridge, not the arm driver). For the real arms: power them on at their control panels (no need to press Play; the driver runs headless), then start the UR driver node from the System view. Shut down with `docker compose down`, not `docker rm -f` (see 9).

8.3 Verify data flow

```
docker exec -it mir_ur_driver bash
ros2 topic list | wc -l          # ≈41
ros2 topic hz /joint_states     # ≈400 Hz, UR driver running
ros2 topic hz /odom             # MiR powered on (see 3.2)
ros2 topic hz /camera/color/image_raw/compressed # ≈30 Hz, camera node running
ros2 control list_controllers
```

8.4 Update

```
git pull
docker restart mir_ui          # backend only (main.py is bind-mounted)
./build.sh mir_ui              # frontend or image changes
docker compose up -d --force-recreate mir_ui
```

9. Troubleshooting

Symptom	Cause	Fix
Topics are listed but carry no messages	/dev/shm not shared with that container	Mount /dev/shm (or /dev) in the service.

Symptom	Cause	Fix
<code>ros2 topic list</code> empty or controllers unresponsive after a hard kill	Stale FastDDS shared-memory segments	Stop the stack, delete <code>/dev/shm/fastrtps_*</code> and <code>sem.fastrtps_*</code> (e.g. via a throw-away container mounting <code>/dev/shm</code>), start again.
Interface reports "no motion"	UR external-control program stopped (protective stop)	The backend resends the program and retries; if it persists, check the teach pendant for a protective stop.
UR driver will not start	Arms were off at launch; stale lock	<code>docker restart mir_ur_driver</code> , then one Start from the interface.
Edit to a mounted script has no effect	Single-file bind mounts keep the old file when an editor replaces it	Restart the container after editing.
Camera at 0 FPS, "no usb"	Camera dropped off the USB bus	Reset USB in the Camera view (needs the host helper installed).
Camera view blank, rosbridge badge red	Arm container not created (rosbridge lives there)	<code>docker compose --profile arms up -d</code> .
Camera container "unhealthy"	No camera node running	Expected; start a camera node from the System view.
MiR panel offline or stale	Base off, Wi-Fi link, or wrong MIR_IP	Check the base, ping MIR_IP, check the Wi-Fi band (3.5).
Disk filling	Recordings in logs/	Delete old bags; recordings are bounded to 10 min and 2 GB files.

10. Testing

- `./run_tests.sh` — Python and Bash syntax checks, Compose validation and unit tests (`--quick` skips the unit tests). On a fresh clone it validates Compose against `config/env.example`. All 19 checks pass (8 October 2026). Before v1.0 a shell bug stopped it after the first check and `--quick` did nothing; both were fixed in v1.0.
- `python3 -m pytest tests/` — 24 backend unit tests (authentication, validation, safety helpers). **Limitation:** they test copies of backend functions instead of importing `main.py`, so they do not detect regressions in the production code.
- Simulation profile (`--profile sim`) — the arm-driver image against simulated hardware, for testing the interface and the API without moving the robots.
- There are no automated integration tests; `tests/test_integration.py`, referenced in `run_tests.sh`, does not exist yet.

11. Known limitations

See Section 7 for security gaps, Section 4.4 for the emergency stop and Section 3.5 for the base. In addition:

- **The images cannot be rebuilt from the repository alone.** MiR/`mir_image/Dockerfile` copies `mir_raw.py`, which was removed; UR/`ur_driver_image/Dockerfile` copies `scripts/ur_entrypoint.sh`, which does not exist; and the UR image is built on `pbd_system:latest` (from Eemil's workspace), whose recipe is not in the repository (`ros2humble_dev_base` is built from `base_image/`). The images on the Jetson are currently the only working copy — do not prune them.
- The backend is a single large file, and the unit tests do not import it.

- The freedrive payload check is interface-only (4.4); the MiR address defaults disagree (3.4); the hand serial ports are not pinned (1).

Open items, owners and priorities are listed in the project report, Section 7.

Appendix A. Environment variables

Set in `config/.env` on the Jetson (never committed); `config/env.example` is a template without secrets. Defaults are those in the code.

Variable	Default	Meaning
ROS_DOMAIN_ID	75	Shared ROS 2 domain
MIR_IP	.14 (backend) / .13 (bridge)	MiR200 address — set explicitly (3.4)
LEFT_ARM_IP / RIGHT_ARM_IP	192.168.1.102 / .103	UR5e arms
NORDBO_LEFT_IP / NORDBO_RIGHT_IP / NORDBO_PORT	.112 / .113 / 2003	Force/torque sensors
UI_PORT / UI_BIND_HOST	8080 / 0.0.0.0	uvicorn; set UI_BIND_HOST=127.0.0.1 behind Caddy
TOKEN_TTL	86400	Session lifetime (s)
LOGIN_MAX_FAILS / LOGIN_LOCKOUT_S	8 / 300	Login throttle
AUTH_USER, AUTH_PASS, WAEL_PASS, PABLO_PASS	insecure defaults	Seed passwords — must be set; warning logged while defaults are in use
SHELL_PASSWORD	empty (terminal disabled)	Second factor for the web terminal
UR_MAX_DELTA	0.5	Maximum jog per move (rad)
UR_STOP_TIMEOUT / UR_JOINTS_TTL	10 / 0.1	Driver stop timeout (s); joint cache (s)
MIR_TIMEOUT / MIR_CACHE_TTL	4.0 / 60.0	MiR REST timeout and cache (s)
PROFILES_DIR / JOINT_SERVER_URL	/app/data/profiles / http://localhost:9091/joints	Internal paths

Appendix B. Repository layout

Each component folder has its own README and documentation/ subfolder. SECURITY.md records the hardening history.

```

docker-compose.yml  7 services; build.sh, start_mir_suite.sh, run_tests.sh
config/             .env (not versioned), tls/ (CA scripts, public certs)
UI/                 backend/, web/, caddy/, mdns/, ui_image/
UR/                 entrypts, start/stop, joint server and mover,
                    action bridge, launch patch
MiR/                bridge_mir/, entrypt, watchdog, documentation/
PERIPHERAL/         camera_orbbec/ (microphones/, speakers/: pending)
ENDEFFECTOR/        BrainCo hands
BATTERY_MS/         phase-2 research notes (also HEAD_NECK/, ROUTER_MODEM/)
documentation/      this document, project report, operator manual
tests/              backend unit tests
robot_workspace/     ROS workspace copy (not versioned)

```